ТОО ”Колледж Хекслет”

Лабораторная работа №2

Полный CRUD Product API с PostgreSQL

Выполнено студентом:

Дубинин Кирилл

Практическое задание

Раздел RO4_3. Полный CRUD Product API с PostgreSQL.

Цель: Закрепить базовые команды SQL и полностью перевести CRUD-операции Product API с JavaScript-массива на PostgreSQL. После выполнения все операции: • Create • Read • Update • Delete должны работать с таблицей products

Условие: RO4_3/lesson2/ЛР-2.pdf

Требования задания

• получать товары через SELECT;

• получать один товар через SELECT ... WHERE;

• создавать товар через INSERT;

• использовать RETURNING после INSERT;

• изменять товар через UPDATE;

• поддерживать частичный PATCH;

• удалять товар через DELETE;

• использовать WHERE для UPDATE и DELETE;

• использовать параметризованные запросы;

• не содержать массива products[];

• хранить SQL только в Service;

• возвращать 404 для отсутствующего id;

• сохранять данные после перезапуска Node.js

Ход выполнения

Массив заменён SELECT, INSERT, UPDATE и DELETE через pg. Пользовательские значения передаются отдельно как параметры $1…$5. RETURNING возвращает изменённую строку или id. В PATCH COALESCE сохраняет поля, которые не были переданы, включая корректное изменение inStock на false. practice.sql демонстрирует фильтрацию, сортировку и изменения в откатываемой транзакции.

Проект: RO4_3/lesson2

Запуск: npm install; npm start. Для PostgreSQL параметры подключения задаются в .env по образцу .env.example. Порядок применения SQL и миграций приведён в README.md проекта.

services/productService.js

services/productService.js

```
import { pool } from '../config/db.js';
export async function getAllProducts(query = {}) {
    const result = await pool.query(`SELECT p.id, p.name, p.price, p.category,
p.in_stock AS "inStock" FROM products p
        WHERE ($1::text IS NULL OR p.category = $1)
        AND ($2::boolean IS NULL OR p.in_stock = $2)
        AND ($3::integer IS NULL OR p.price >= $3)
        AND ($4::integer IS NULL OR p.price <= $4) ORDER BY p.id`,
        [query.category ?? null, query.inStock ?? null, query.minPrice ?? null,
query.maxPrice ?? null]);
    return result.rows;
}
export async function findProductById(id) {
    const result = await pool.query(`SELECT p.id, p.name, p.price, p.category,
p.in_stock AS "inStock" FROM products p WHERE p.id = $1`, [id]);
    return result.rows[0];
}
export async function addProduct(data) {
    const result = await pool.query(`INSERT INTO products (name, price, category,
in_stock)
        VALUES ($1, $2, $3, $4) RETURNING id`, [data.name, data.price, data.category,
data.inStock]);
    return findProductById(result.rows[0].id);
}
export async function updateProductById(id, changes) {
    const result = await pool.query(`UPDATE products SET name = COALESCE($1, name),
price = COALESCE($2, price),
        category = COALESCE($3, category), in_stock = COALESCE($4, in_stock) WHERE id =
$5 RETURNING id`,
        [changes.name ?? null, changes.price ?? null, changes.category ?? null,
changes.inStock ?? null, id]);
    return result.rows[0] ? findProductById(id) : null;
}
export async function deleteProductById(id) {
    const result = await pool.query(`DELETE FROM products WHERE id = $1 RETURNING id,
```

Листинг 1. services/productService.js (часть 1)

services/productService.js

```
name, price, category, in_stock AS "inStock"`, [id]);
    return result.rows[0];
}
```

Листинг 2. services/productService.js (часть 2)

sql/practice.sql

```sql
-- Все изменения учебного примера отменяются в конце.
BEGIN;
SELECT id, name, price FROM products WHERE price > 10000 AND in_stock = true ORDER BY price ASC;
UPDATE products SET price = 20000 WHERE id = (SELECT MAX(id) FROM products) RETURNING *;
DELETE FROM products WHERE id = (SELECT MAX(id) FROM products WHERE name = 'Webcam') RETURNING *;
ROLLBACK;
```

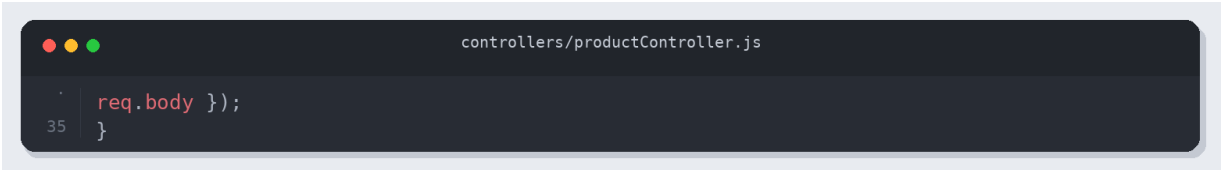
Листинг 3. sql/practice.sql

controllers/productController.js

```js
import * as service from '../services/productService.js';
import { AppError } from '../errors/AppError.js';

export async function getProducts(req, res, next) {
    try { res.json(await service.getAllProducts(req.query)); }
    catch (error) { next(error); }
}
export async function getProductById(req, res, next) {
    try {
        const product = await service.findProductById(Number(req.params.id));
        if (!product) return next(new AppError('Product not found', 404));
        res.json(product);
    } catch (error) { next(error); }
}
export async function createProduct(req, res, next) {
    try { res.status(201).json(await service.addProduct(req.body)); }
    catch (error) { next(error); }
}
export async function updateProduct(req, res, next) {
    try {
        const product = await service.updateProductById(Number(req.params.id), req.body);
        if (!product) return next(new AppError('Product not found', 404));
        res.json(product);
    } catch (error) { next(error); }
}
export async function deleteProduct(req, res, next) {
    try {
        const product = await service.deleteProductById(Number(req.params.id));
        if (!product) return next(new AppError('Product not found', 404));
        res.json(product);
    } catch (error) { next(error); }
}
export function productSettings(req, res) {
    res.json({ id: Number(req.params.id), notify: req.query.notify === 'true', changes:
```

Листинг 4. controllers/productController.js (часть 1)



Листинг 5. controllers/productController.js (часть 2)

Проверка результата

Проверено: 05.11.2026 13:38 (Алматы). Node.js v26.12.1.

Приведены фактические результаты проверки. Полный журнал: verification.json.

• Реальный PostgreSQL

• HTTP endpoints

• CRUD сохраняется в PostgreSQL

HTTP-запросы

GET /api/products → 200

GET /api/products/1 → 200

GET /api/products/999999 → 404

GET /api/products/abc → 400

GET /api/products?category=Accessories&inStock=true&minPrice=10000&maxPrice=50000 → 200

GET /api/products?minPrice=abc → 400

GET /api/products?inStock=maybe → 400

GET /api/products?minPrice=500&maxPrice=100 → 400

POST /api/products → 403

POST /api/products → 400

POST /api/products → 201

PATCH /api/products/6 → 200

DELETE /api/products/6 → 200

GET /api/products/6 → 404

GET /api/unknown → 404

Примеры ответов

POST /api/products → 201


```
{
  "id": 6,
  "name": "Test SSD",
  "price": 65000,
  "category": "Accessories",
  "inStock": true
}
```


PATCH /api/products/6 → 200


```
{
```

```
  "id": 6,
  "name": "Test SSD",
  "price": 42000,
  "category": "Accessories",
  "inStock": false
}
```


DELETE /api/products/6 → 200


```
{
  "id": 6,
  "name": "Test SSD",
  "price": 42000,
  "category": "Accessories",
  "inStock": false
}
```


Результат

Проверены обязательные операции и указанные ошибочные сценарии; результаты сохранены вместе с исходным кодом.